STUDENT'S PERSONAL RECORD

Name Adwika Vishal

Class / Sec. _____ Roll No. _____ Subject _____

Class Teacher _____ Subject Teacher _____

INDEX

TELCO

Page No.
Date 24 2 25

Q1 Write a C program to find the sum of rows and columns of a matrix

A-1
```
#include <stdio.h>
int main(){
    int matrix[10][10];
    int rows, cols;
    int i, j;
    int rowSum, colSum;
    printf("Enter no. of rows: ");
    scanf("%d", &rows);
    printf("Enter no. of cols: ");
    scanf("%d", &cols);
    printf("Enter matrix elements");
    for(int i=0; i<rows; i++){
        for(int j=0; j<cols; j++){
            scanf("%d", &matrix[i][j]);
    }}
    printf("\nSum of each row:\n");
    for(int i=0; i<rows; i++){
        rowSum=0;
        for(int j=0; j<cols; j++){
            rowSum += matrix[i][j];
        printf("Row %d sum = %d\n",
                i+1, rowSum);
    }
```

```
printf("\n Sum of each column: \n");
for(j=0; j<cols; j++){
    colSum = 0;
    for(i=0; i<rows; i++){
        colSum += matrix[i][j];
    printf("Colum %d sum = %d \n",
        j+1, colSum); }
return 0;
}
```

Output

```
Enter no. of rows: 2
Enter no. of cols: 3
Enter matrix elements:
1 2 3
4 5 6

Sum of each row:
Row 1 sum = 6
Row 2 sum = 15

Sum of each column:
Column 1 sum = 5
Column 2 sum = 7
Column 3 sum = 9
```

Q2 Write a C program to count duplicate elements in an array

A-2

```
#include <stdio.h>
int main(){
    int arr[100];
    int n;
    int i, j;
    int count = 0;
    printf("Enter size of array: ");
    scanf("%d", &n);
    printf("Enter array elements: \n");
    for(i=0; i<n; i++){
        scanf("%d", &arr[i]); }
    for(i=0; i<n; i++){
        for(j=i+1; j<n; j++){
            if(arr[i]==arr[j]){
                count++;
                break; } } }
    printf("Total no. of duplicate
    elements = %d \n", count);
    return 0;
}
```

Output

Enter size of array: 6
Enter array elements:
1 2 3 2 4 1
Total no. of duplicate elements = 2

Q3 Write a C program for First Come First Serve Scheduling

A-3

```
#include <stdio.h>
int main(){
    int n, i;
    printf("Enter no. of processes");
    scanf("%d", &n);
    int bt[n], wt[n], tat[n];
    printf("Enter Burst time for
    each process: \n");
    for(i=0; i<n; i++){
        printf("Process %d:", i+1);
        scanf("%d", &bt[i]); }
    wt[0]=0;
    for(i=1; i<n; i++){
        wt[i]=wt[i-1]+bt[i-1];
    }
    for(i=0; i<n; i++){
        tat[i]=wt[i]+bt[i];
    }
    printf("\n Process\t Burst time
    \tWaiting Time \t Turnaround
    Time\n");
    for(i=0; i<n; i++){
        printf("P%d\t%d\t\t%d
        \t\t%d\n", i+1, bt[i], wt[i], tat[i]
}}
```

~~Output~~

```
Enter no. of processes : 3
Enter Burst time for each process:
Process 1 : 5
Process 2 : 3
Process 3 : 8
```

Process	bt	wt	TT
P1	5	0	5
P2	3	5	8
P3	8	8	16

Page No.
Date 3 3 26

Q4 Write a C program for Shortest Job First Scheduling

A-4

```
#include <stdio.h>
int main(){
    int n, i, j;
    int bt[20], wt[20], ta[20], p[20];
    float avg_wt = 0, avg_ta = 0;
    printf("Enter no. of processes : ");
    scanf("%d", &n);
    for(i=0; i<n; i++){
        printf("Enter Burst Time %d", i+1);
        scanf("%d", &bt[i]);
        p[i] = i+1;
    }
    for(i=0; i<n; i++){
        for(j=i+1; j<n; j++){
            if(bt[i] > bt[j]){
                int temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    wt[0] = 0;
    for(i=1; i<n; i++){
        wt[i] = wt[i-1] + bt[i-1];
```

```
for(i=0;i<n;i++){
    tat[i] = wt[i] + bt[i];}
printf("\nProcess \t Burst Time\t
Waiting Time \t Turnaround Time\n");
for(i=0;i<n;i++){
    printf("P%d \t %d\t\t %d\t\t %d\n",
    p[i], bt[i], wt[i], tat[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];}
    avg_wt /= n;
    avg_tat /= n;
    printf("\n Average Monitoring
Time = %.2f", avg_wt);
    printf("\n Average Turnaround
Time = %.2f\n", avg_tat);
    return 0;
}
```

Output

```
Enter no. of processes: 4
Enter Burst Time for Process 1: 6
Enter Burst Time for Process 2: 8
Enter Burst Time for Process 3: 7
Enter Burst Time for Process 4: 3
```

Process	Burst Time	Waiting Time	Turnaround
P4	3	0	3
P1	6	3	9
P3	7	9	16
P2	8	16	24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00

Page No.
Date 16 3 26

Q1 Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time
Priority (Pre.-emptive)

Algorithm

1. Read no. of processes
2. Input burst time and priority
3. Sort processes based on priority
4. Calculate
   - Waiting Time = sum of previous burst
   - Turnaround Time = Waiting Time + Burst Time

Code

```
#include <stdio.h>
int main(){
    int n, i, j;
    int bt[20], wt[20], tat[20], pr[20],
    p[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    for(i=0;i<n;i++){
        printf("\n Process %d \n", i+1);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
```

```
scanf("%d", &pr[i]);
p[i] = i+1; }
for(int i=0; i<n; i++){
  for(j=i+1; j<n; j++){
    if(pr[i] > pr[j]){
      int temp;
      temp = pr[i];
      pr[i] = pr[j];
      pr[j] = temp;
      temp = bt[i];
      bt[i] = bt[j];
      bt[j] = temp;
      temp = p[i];
      p[i] = p[j];
      p[j] = temp; } } }
wt[0] = 0;
for(i=1; i<n; i++)
  wt[i] = wt[i-1] + bt[i-1];
for(i=0; i<n; i++)
  tat[i] = wt[i] + bt[i];
printf("\n Process \t Priority \t
Burst \t Waiting \t Turnaround \n");
for(i=0; i<n; i++){
  printf("P%d \t %d \t %d \t
  %d \t %d \n", p[i], pr[i],
  bt[i], wt[i], tat[i]);
  avg_wt += wt[i];
  avg_tat += tat[i]; }
```

```
printf("\n Average Waiting Time = %.2f",
  avg_wt/n);
printf("\n Average Turnaround
Time = %.2f \n", avg_tat/n);
return 0; }
```

Output

Enter no. of processes: 3
Process 1
Burst Time: 10
Priority: 2

Process 2
Burst Time: 5
Priority: 1

Process 3
Burst Time: 8
Priority: 3

Process	Priority	Burst	Waiting	Turnaround
P2	1	5	0	5
P1	2	10	5	15
P3	3	8	15	23

Average Waiting Time = 6.67
Average Turnaround Time = 14.33

Preemphive

Code

~~# int n, i, time = 0, completed = 0;~~

```
#include<stdio.h>
int main(){
    int n, i, time=0, completed=0;
    int bt[20], rt[20], pr[20];
    int wt[20], tat[20], at[20];
    float avg_wt=0, avg_tat=0;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    for(i=0; i<n; i++){
        printf("\n Process %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
        scanf("%d", &pr[i]);
        rt[i]=bt[i];
    }
    while(completed!=n){
        int highest=-1;
        int min_priority=9999;
        for(i=0; i<n; i++){
            if(at[i]<=time && rt[i]>0
&& pr[i]<min_priority){
                min_priority=pr[i];
                highest=i;
            }
        }
        if(highest==-1){
            time++;
            continue;
        }
        rt[highest]--;
        time++;
        if(rt[highest]==0){
            completed++;
            int finish=time;
            tat[highest]=finish-at[highest];
            wt[highest]=tat[highest]-
                bt[highest];
            avg_wt+=wt[highest];
            avg_tat+=tat[highest];
        }
    }
    printf("\n Average Waiting Time=
%.2f", avg_wt/n);
    printf("\n Average Turnaround
Time=%.2f\n", avg_tat/n);
    return 0;
}
```

Output

```
Enter no. of processes: 3
Process 1
Arrival Time: 0
Burst Time: 5
Priority: 2
```

Page No.
Date

Process 2
Arrival Time: 1
Burst Time: 3
Priority: 1

Process 3
Arrival Time: 2
Burst Time: 4
Priority: 3

Process	Waiting	Turnaround
P1	3	8
P2	0	3
P3	6	10

Average Waiting Time = 3.00
Average Turnaround Time = 7.00

Page No.
Date 17 3 26

Round Robin

```
#include<stdio.h>
int main(){
  int n,tq,i;
  printf("Enter no. of processes:");
  scanf("%d",&n);
  int bt[n],rem_bt[n],wt[n],
  tat[n];
  for(i=0;i<n;i++){
    printf("Enter Burst Time
    for P%d:",i+1);
    scanf("%d",&bt[i]);
    rem_bt[i]=bt[i];}
  printf("Enter time quantum");
  scanf("%d",&tq);
  int time=0,done;
  while(1){
    done=1;
    for(i=0;i<n;i++){
    if(rem_bt[i]>0){
      done=0;
      if(rem_bt[i]>tq){
        time+=tq;
        rem_bt[i]-=tq;}
      else{
        time+=rem_bt[i];
        wt[i]=time-bt[i];
        rem_bt[i]=0;}}}
```

```
if (done == 1)
    break;
float avg_wt = 0, avg_tat = 0;
printf("\n Process \t Burst Time
\t Waiting Time \t TAT \n");
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
    total_wt += wt[i];
    total_tat += tat[i];
    printf("\n Average
Waiting Time = %.2f", total_wt/n);
    printf("\n Average
Turnaround Time = %.2f\n",
    total_tat/n);
    return 0;
}
```

output

Enter no. of processes: 3
Enter Burst Time for P1: 2
Enter Burst Time for P2: 6
Enter Burst Time for P3: 4
Enter Time Quantum: 2

Process	BT	WT	TT
P1	2	0	2
P2	6	6	12
P3	4	6	10

Average Waiting Time = 4.00

Average Turnaround Time: 8.00

Gantt Chart:

P1	P2	P3	P2	P3	P2	
0	2	4	6	8	10	12

17/1/25

Enter no. of processes: 3
Enter Burst time for P1: 2
Enter Burst time for P2: 6
Enter Burst time for P3: 4
Enter Time quantum: 3

Process	BT	WT	TAT
P1	2	0	2
P2	6	5	11
P3	4	8	12

Average Waiting Time = 4.33
Average Turnaround Time = 8.33
Gantt Chart

P1	P2	P3	P2	P3	
0	2	5	8	11	12

Smaller TQ → improves responsiveness and reduces waiting time
Larger TQ → Increases waiting time

Multi-level Queue

Code →

```
#include<stdio.h>
#define MAX 100
typedef struct {
  int id, at, bt, rt, ct, wt, tat, type;
} Process;

Process p[MAX];
int n;
int sysQ[MAX], userQ[MAX];
int frontS = 0, rearS = 0;
int frontU = 0, rearU = 0;

void enqueue(int q[], int *rear,
             int val) {
  q[(*rear)++] = val;
}

int dequeue(int q[], int *front,
            int *rear) {
  if (*front == *rear) return -1;
  return q[(*front)++];
}

int isEmpty(int front, int rear) {
  return front == rear;
}

int main() {
  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for(int i = 0; i < n; i++) {
    printf("\nProcess %d\n", i);
    printf("Enter arrival time: ");
    scanf("%d", &p[i].at);
    printf("Enter burst time: ");
    scanf("%d", &p[i].bt);
    printf("Enter type (0=system,
      1=user): ");
    scanf("%d", &p[i].type);

    p[i].id = i;
    p[i].rt = p[i].bt; }

  for(int i = 0; i < n-1; i++) {
    for(int j = i+1; j < n; j++) {
      if (p[i].at > p[j].at) {
        Process temp = p[i];
        p[i] = p[j];
        p[j] = temp; } } }

  int time = 0, completed = 0, i = 0;
  int current = -1;
  int gantt[MAX], gtime[MAX],
    gindex = 0;
```

```
while (completed < n) {
  while (i<n && p[i].at <= time) {
    if (p[i].type == 0)
      enqueue(sysQ, &rearS, i);
    else
      enqueue(userQ, &rearU, i);
    i++; }

  if (current != -1 && p[current].
type == 1 && !isEmpty(&frontS,
&rearS)) {
    enqueue(userQ, &rearU,
        current);
    current = -1; }
  if (current == -1) {
    if (!isEmpty(&frontS, &rearS))
      current = dequeue(sysQ,
      &frontS, &rearS);
    else if (!isEmpty(&frontU,
    &rearU))
      current = dequeue(userQ,
      &frontU, &rearU);
    else {
      time++;
      continue; }}

ganttChart[gIndex] = p[current].id;
gTime[gIndex] = time;
gIndex++;
```

```
p[current].rt--;
time++;
if (p[current].rt == 0) {
  p[current].ct = time;
  p[current].tat = p[current].ct
- p[current].at;
  p[current].wt = p[current].tat
- p[current].bt;
  completed++;
  current = -1; }}
gTime[gIndex] = time;

printf("\nGantt Chart:\n");
for(int i=0; i<gIndex; i++) {
  printf("P%d|", ganttChart[i]); }

printf("\n0");
for(int i=0; i<gIndex; i++) {
  printf(" %d", gTime[i+1]); }

float totalWT=0, totalTAT=0;

printf("\nID\tType\tAT\tBT
\tCT\tWT\tTAT\n");
for(int i=0; i<n; i++) {
  printf("%d\t%s\t%d\t%d
\t%d\t%d\t%d\n",
    p[i].id,
```

```
(p[i].type == 0) ? "System" :
"User",
p[i].at, p[i].bt, p[i].ct,
p[i].wt, p[i].tat);

totalWT += p[i].wt;
totalTAT += p[i].tat; 3

printf("\nAverage Waiting Time:
%.2f", totalWT/n);
printf("\n Average Turnaround
Time: %.2f\n", totalTAT/n);

return 0; 3
```

Output →

```
Process 0
Enter arrival time: 1
Enter burst time: 2
Enter type [0 = System, 1 = User]: 0

Process 1
Enter arrival time: 3
Enter burst time: 4
Enter type (0 = System, 1 = User): 1

Process 2
Enter arrival time: 3
Enter burst time: 4
Enter type (0 = System, 1 = User): 0
```

Gantt Chart:

P0	P0	P2	P2	P2	P2	P1	P1	P1	P1
0	2	3	4	5	6	7	8	9	10	11

ID	Type	AT	BT	CT	WT	TAT
0	System	1	2	3	0	2
1	User	3	4	11	4	8
2	System	3	4	7	0	4

Average Waiting Time: 1.33
Average Turnaround Time: 4.67

24/12/25

Rate Monotonic and Earliest deadline first

```
#include <stdio.h>
#include <math.h>
#define MAX 20

typedef struct {
    int id;
    int burst;
    int deadline;
    int period;
    int share;
    int waiting;
    int turnaround;
    int completion; } Process;

void edf(Process p[], int n) {
    Process temp;
    float util = 0;
    for(int i=0; i<n; i++)
        util += (float)p[i].burst/
            p[i].deadline;
    printf("\nEDF utilization =
    %.2f", util);
    printf(util <= 1 ? "\nEDF:
    SCHEDULABLE\n": "\nEDF:
    NON-SCHEDULABLE\n");
```

```
for(int i=0; i<n-1; i++) {
    for(int j=i+1; j<n; j++) {
        if(p[i].deadline > p[j].deadline) {
            temp = p[i];
            p[i] = p[j];
            p[j] = temp; } } }

int time=0;
printf("\nEarliest Deadline First (EDF)
\n");
printf("ID\tBT\tDeadline\tCT\t
WT\tTAT\n");

for(int i=0; i<n; i++) {
    p[i].waiting = time;
    time += p[i].burst;
    p[i].completion = time;
    p[i].turnaround = time;

    printf("%d\t%d\t%d\t\t%d
    \t%d\t%d\n",
    p[i].id, p[i].burst, p[i].deadline,
    p[i].completion, p[i].waiting,
    p[i].turnaround; } }
```

```
float calculate_utilization(Process p[], int n,
    int use_period) {
    float utilization = 0
    for (0 to n) {
        utilization = (float)p[i].burst/(use_
        period ? p[i].period : p[i].deadline);
    return utilization;
}
void rms(Process p[], int n) {
    Process temp;
    float ubi = 0;
    for (int i = 0; i < n; i++)
        ubi += (float)p[i].burst/p[i].
        period;
    float u = calculate_utilization(p, n, 1);
    float limit = n * (pow(2, float)(1/n)
    - 1);
    printf("\nRMS Utilization =
%0.2f", ubi);
    printf("\nRMS Limit = %0.2f",
    limit);
    printf(ubi <= limit ? "\nRMS:
    SCHEDULABLE\n" : "\nRMS:
    NON-SCHEDULABLE\n");

    for(int i=0; i<n-1; i++){
        for(int j=i+1;

    if (u > 1.0) {
        printf("\nRESULT: DEFINITELY
NOT SCHEDULABLE (Utilization > 1)\n");
        return; }
    else if (u > limit) {
        printf("\nRESULT: POTENTIALLY
NOT SCHEDULABLE (Exceeds
Liu-Layland Bound)\n"); }
```

```
    else {
        printf("\nRESULT: SCHEDULABLE\n");
    }
    Process temp;
    for(int i=0; i<n-1; i++) {
        for(int j=i+1; j<n; j++) {
            if (p[i].period > p[j].period) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp; 333
    int time = 0;
    printf("ID\tBT\tPRD\tCT\tWT
    \tTAT\n");
    for(int i=0; i<n; i++) {
        p[i].waiting = time;
        time += p[i].burst;
        p[i].completion = time;
        p[i].turnaround = p[i].completion
    }}
void proportional(Process p[], int n)
    int total_shares = 0;
    for(int i=0; i<n; i++)
        total_shares += p[i].share;
    printf("\n--- Proportional
    Share Scheduling ---");
    printf("\nTotal Shares Allocated
    %d", total_shares);
```

```
if (total_shares > 100) {
    printf("\nRESULT: OVERLOADED
    (Total share > 100)%%)\n");
    else {
        printf("\nRESULT:
        SCHEDULABLE\n"); }
int time = 0;
printf("ID\tBT\tshare\tCT
\tWT\tTAT\n");
for (int i=0; i<n; i++) {
    p[i].waiting = time;
    time += p[i].burst;
    p[i].completion = time
    p[i].turnaround = p[i].
    completion; }}

int main() {
    int n;
    Process p[MAX], p1[MAX],
    p2[MAX], p3[MAX];

    printf("Enter no. of processes:");
    scanf("%d", &n);
    for(int i=0; i<n; i++) {
        p[i].id = i+1;
        printf("\nProcess %d (Burst,
Deadline, Period Share): ", i+1);
        scanf("%d %d %d %d",
&p[i].burst, &p[i].deadline, &p[i].
period, &p[i].share);
        p1[i] = p2[i] = p3[i] = p[i]; }

    qsort(p2, n);
    edf(p1, n);
    proportional(p3, n);
    return 0; }
```

Output

```
Enter no. of processes: 2
Process 1: 3 8 1 6
Process 2: 3 7 2 4

---Rate Monotonic Scheduling---
Total Utilization: 4.5 (Limit: 0.83)
RESULT: Definitely NOT SCHEDULABLE
(Utilization > 1)

---Earliest Deadline First---
Total Utilization: 0.8
RESULT: SCHEDULABLE
ID  BT  DL  CT  WT  TAT
2   3   7   3   0   3
1   3   8   6   3   6
```

$U = \sum \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$

Process 1 = 1 4 4 30
Process 2 = 1 5 5 30
Process 3 = 2 20 20 40

$3(2^{1/3} - 1) = 0.78$

$0.55 < 0.78$

SCHEDULABLE

ID	BT	PRD	CT	WT	TAT
1	1	4	4	6	1
2	1	5	2	1	2
3	2	20	4	2	4

Q1 The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as S0 = 1, S1 = 0, S2 = 0. Find out how many times Process P0 will print "0". Assume order of execution as P0, P1, P2, P0, P1

```
Process P0:
while(true){
    wait(S0);
    printf("0");
    signal(S1);
    signal(S2); }
Process P1:
    wait(S1);
    signal(S0);
Process P2:
    wait(S2);
    signal(S0);
```

Page No.
Date

Ans

Initial	S0=1	S1=0	S2=0	Remark
P_0 →	0	1	1	0
P_1 →	1	0	1	—
P_2 →	1	0	0	—
P_0 →	0	1	1	00
P_1 →	1	0	1	—

P0 will print 0 two times

Q2 Consider two processes A and B. Two semaphore variable S & T are used to synchronize the processes. S is initialized to 0 and T is initialized to 1. Processes are scheduled →
A B A B B A A B A

Process A:	Process B:
while (1) {	while (1) {
wait(S);	wait(T);
print 'P';	print 'C';
print 'P';	print 'I';
signal(T);	signal(S);
}	}

Page No.
Date

Ans

Initial	S = 0	T = 1	Remark
A	0	1	Process A is blocked
B	1	0	II
A	0	1	II PP
B	1	0	II PP II
B	1	0	Process B is blocked
A	0	1	II PP II PP
A	0	1	Process A is blocked
B	1	0	II PP II PP I
A	0	1	II PP II P P II PP

Bounded Buffer

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>

int *buffer;
int BUFFER_SIZE;

int in=0; out=0;
int total_items;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void*arg){
    for(int i=0; i<total_items; i++){
        int item = i+1;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("[Producer] Produced:
%d -> Buffer[%d]\n", item, in);
        in = (in+1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1); }
    return NULL; }
```


```
void* consumer(void* arg){
    for(int i=0; i<total_items; i++){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("[Consumer] Consumed:
%d <- Buffer[%d]\n", item,
        out);
        out = (out+1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2); }
    return NULL;
}
int main(){
    pthread_t p, c;
    printf("Enter buffer size: ");
    scanf("%d", &BUFFER_SIZE);
    printf("Enter no. of items: ");
    scanf("%d", &total_items);
    buffer = (int*)malloc(sizeof(int)
*BUFFER_SIZE);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&p, NULL,
    producer, NULL);
    pthread_create(&c, NULL,
    consumer, NULL);
```

```
pthread_join(p, NULL);
pthread_join(c, NULL);
free(buffer);
return 0;
}
```

Output

```
Enter buffer size: 3
Enter no. of items: 10
[Producer] Produced: 1 -> Buffer[0]
[Consumer] Consumed: 1 <- Buffer[0]
[Producer] Produced: 2 -> Buffer[1]
[Consumer] Consumed: 2 <- Buffer[1]
[Producer] Produced: 3 -> Buffer[2]
[Consumer] Consumed: 4 <- Buffer[0]
[Producer] Produced: 4 -> Buffer[0]
[Consumer] Consumed: 3 <- Buffer[2]
[Producer] Produced: 5 -> Buffer[1]
[Producer] Produced: 6 -> Buffer[2]
[Consumer] Consumed: 4 <- Buffer[0]
[Producer] Produced: 7 -> Buffer[0]
[Consumer] Consumed: 5 <- Buffer[1]
[Producer] Produced: 8 -> Buffer[1]
[Consumer] Consumed: 6 <- Buffer[2]
[Producer] Produced: 9 -> Buffer[2]
[Consumer] Consumed: 7 <- Buffer[0]
[Producer] Produced: 10 -> Buffer[0]
[Consumer] Consumed: 8 <- Buffer[1]
[Consumer] Consumed: 9 <- Buffer[2]
[Consumer] Consumed: 10 <- Buffer[0]
```

~~Dining Philosopher~~

		in	out	empty	full
0	[, ,]	0	0	3	0
1	[1, ,]	1	0	2	1
2	[1, -,]	1	1	3	0
3	[-, 2, -]	2	1	2	1
4	[-, 2, 3]	0	1	1	2
5	[-, -, 3]	0	2	2	1
6	[4, -, 3]	1	2	1	2
7	[4, -, -]	1	0	2	1
8	[4, 5, -]	2	0	1	2
9	[-, 5, -]	2	1	2	1
10	[-, -, -]	2	2	3	0

Dining Philosopher Problem

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>
#define N 5
sem_t room;
sem_t chopstick[N];
pthread_t int N;
void* dine(void *arg){
    int id = *(int*)arg;
    while(1){
        printf("Philosopher %d is
THINKING \n", id);
        sleep(1);
        sem_wait(&room);
        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id+1)%
N]);
        printf("Philosopher %d is
EATING \n", id);
        sleep(1);
        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id+1)]
%N]);
        sem_post(&room);
    }
}
```

```
int main(){
    int i;
    printf("Enter no. of philosophers");
    scanf("%d", &N);
    pthread_t philosopher[N];
    int ids[N];
    chopstick = (sem_t*)malloc(N*
    sizeof(sem_t));
    sem_init(&room, 0, N-1);
    for(i=0; i<N; i++){
        sem_init(&chopstick[i], 0, 1);
    }
    for(i=0; i<N; i++){
        ids[i]=i;
    pthread_create(&philosopher[i],
    NULL, dine, &ids[i]); }

    for(i=0; i<N; i++){
        pthread_join(philosopher[i],
        NULL); }

    return 0;
}
```

Output

Enter number of philosophers: 3
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 2 is eating
Philosopher 2 is finished eating
Philosopher 2 is thinking
Philosopher 1 is eating
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 2 is eating
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 1 is eating
Philosopher 1 finished eating
Philosopher 2 is eating
Philosopher 2 finished eating

Output Table

step	P0	P1	P2	P3	P4
1	1	1	1	1	1
2	0	1	0	1	1
3	1	0	1	0	1
4	0	1	1	1	0
5	1	0	0	1	1
6	1	1	1	0	0
7	0	1	0	1	1
8	1	0	1	0	1

28/4/22

Page No.
Date

Bankers' Algorithm

Safety Algorithm

1. Let Work and Finish be vectors of length m & n respectively. Initialize:
   Work = Available
   Finish[i] = false for i = 0, 1 ... n-1
2. Find an i such that both:
   (a) Finish[i] = false
   (b) Need_i ≤ Work
   If no such i exists go to step 4
3. Work = Work + Allocation
   Finish[i] = true
   go to step 2
4. Finish[i] == true for all i, then the system is in a safe state

Q1 Consider the following matrix where max resources A, B, C → 10, 5, 7

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1
	7 2 5			

Page No.
Date

A1 ≡

	A	B	C	
	10	5	7	{P_1, P_3, P_4, P_0, P_2}
	7	2	5	
	3	3	2	

Need = Max - Allocation
Available = Allocation Total - Allocated resources

P_0 Finish[0] = false
Need ≤ available
743 ≰ 332

P_1 Finish[1] = false
122 ≤ 332
Work = Work + Allocation
= 332 + 200
= 532
Finish[1] = true

P_2 Finish[2] = false
600 ≰ 532

P_3 Finish[3] = false
011 ≤ 532
Work = 211 + 532
= 743
Finish[3] = true

P_4 Finish[4] = false
431 ≤ 743
Work = Work + Allocation
= 743 + 002
= 745 Finish[4] = true

P0 743 < 745
Work = 745 + 010
= 755
Finish[0] = true

P2 600 ≤ 755
Work = 755 + 302
= 1057
Finish[2] = true

Code

```
#include <stdio.h>
int main(){
  int n, m, i, j, k;
  printf("Enter no. of processes:");
  scanf("%d", &n);

  printf("Enter no. of resources:");
  scanf("%d", &m);
  int alloc[n][m], max[n][m],
  need[n][m];
  int total[m], avail[m];

  printf("Enter Allocation Matrix:\n");
  for(i=0; i<n; i++){
    for(j=0; j<m; j++){
      scanf("%d", &alloc[i][j]);
    }
  }

  printf("Enter Max Matrix:\n");
  for(i=0; i<n; i++){
    for(j=0; j<m; j++){
      scanf("%d", &max[i][j]);
    }
  }
  printf("Enter Total Resources:\n");
  for(i=0; i<m; i++){
    scanf("%d", &total[i]);
  }

  for(i=0; i<m; i++){
    int sum=0;
    for(j=0; j<n; j++){
      sum += alloc[j][i];
    }
    avail[i] = total[i] - sum;
  }

  for(i=0; i<n; i++){
    for(j=0; j<m; j++){
      need[i][j] = max[i][j] -
        alloc[i][j];
    }
  }
  printf("\nProcess\tAllocation\tMax
    \tNeed\n");
  for(i=0; i<n; i++){
    printf("P%d\t", i);
    for(j=0; j<m; j++){
      printf("%d", alloc[i][j]);
    }
    printf("\t");
    for(j=0; j<m; j++){
      printf("%d", max[i][j]);
    }
    printf("\t");
```

```
        for(j=0; j<m; j++)
            printf("%d", need[i][j]);
        printf("\n");
    }
    printf("\nAvailable Resources :");
    for(i=0; i<m; i++) {
        printf("%d", avail[i]);
    }
    printf("\n");
    int finish[n], safeseq[n];
        count=0;
    for(i=0; i<n; i++)
        finish[i]=0;
    int work[m];
    for(i=0; i<m; i++)
        work[i]=avail[i];
    while(count<n){
        int found=0;
        for(i=0; i<n; i++){
            if(finish[i]==0){
                int flag=1;
                for(j=0; j<m; j++){
                    if(need[i][j]>work[j]){
                        flag=0;
                        break;
                    }
                }
                if(flag){
                    for(k=0; k<m; k++){
                        work[k]+=alloc[i][k];
                    }
                    safeseq[count++]=i;
                    finish[i]=1;
                    found=1;
                }
            }
        }
        if(found==0){
            printf("\nSystem is NOT in safe state\n");
            return 0;
        }
    }
    printf("\nSystem is in SAFE state");
    printf("\nSafe Sequence");
    for(i=0; i<n; i++){
        printf("P%d", safeseq[i]);
    }
    printf("\n");

    return 0;
}
```

Output

```
Enter no. of processes: 5
Enter no. of resources: 4
Enter Allocation Matrix:
0 0 1 2
1 0 0 0
1 3 5 4
0 6 3 2
0 0 1 4
```

Enter Max Matrix:

0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

Enter Available Resources:
3 14 12 12

Process	Allocation	Max	Need
	A B C D	A B C D	A B C D
P0	0 0 1 2	0 0 1 2	0 0 0 0
P1	1 0 0 0	1 7 5 0	0 7 5 0
P2	1 3 5 4	2 3 5 6	1 0 0 2
P3	0 6 3 2	0 6 5 2	0 0 2 0
P4	0 0 1 4	0 6 5 6	0 6 4 2

System is in safe state
Safe Sequence: P0 P2 P3 P4 P1

P0 → Need = [0 0 0 0] ≤ Work[1 5 2 0]
Work = Work + Alloc
= 1 5 3 2
Finish[0] = true

P1 → (0 7 5 0) ≰ (1 5 3 2)

P2 → Need = 1 0 0 2 ≤ 1 5 3 2
Work = 1532 + 1354
= 2 8 8 6
Finish[2] = true

P3 → Need = 0 0 2 0 ≤ 2 8 8 6
Work = 2 8 8 6 + 0 6 3 2
= 2 14 11 8
Finish[3] = true

P4 → Need = 0 6 4 2 ≤ 2 14 11 8
Work = 2 14 11 8 + 0 0 1 4
= 2 14 12 12
Finish[4] = true

P1 → Need = 0 7 5 0 ≤ 2 14 12 12
Work = 2 14 12 12 + 1 0 0 0
= 3 14 12 12
Finish[1] = true.

Seq" → P0, P2, P3, P4, P1

28/4/24

Page No.
Date

Resource Request Algorithm

```
int p;
printf("Enter process no. making
requests:");
scanf("%d", &p);

int request[m];
printf("Enter request vector:\n");
for(i=0; i<m; i++)
    scanf("%d", &request[i]);

for(i=0; i<m; i++){
  if(request[i] > need[p][i]){
      printf("Error: Request exceeds
      need\n");
      return 0; }
}
for(i=0; i<m; i++){
  if(request[i] > avail[i]){
    printf("Resources not available,
  process must wait\n");
    return 0; }
}
for(i=0; i<m; i++){
  avail[i] -= request[i];
  alloc[p][i] += request[i];
  need[p][i] -= request[i]; }
```

Page No.
Date

```
int finish[n], safeseq[n];
for(i=0; i<n; i++){
    finish[i]=0;
}
```

Output

```
Enter no. of processes: 5
Enter no. of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter available resources:
3 3 2
Enter process no. making request: 1
Enter request vector:
1 0 2

System is in safe state
Safe seq: P1 P3 P4 P0 P2
```

Deadlock Detection

```
#include<stdio.h>
int main(){
    int n, m, i, j, k;
    printf("Enter no. of processes:");
    scanf("%d", &n);

    printf("Enter no. of resources:");
    scanf("%d", &m);
    int alloc[n][m], request[n][m],
    avail[m];
    printf("Enter Allocation matrix:");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter request matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &request[i][j]);

    printf("Enter Available Resources:\n");
    for(i=0; i<m; i++)
        scanf("%d", &avail[i]);

    int finish[n];

    for(i=0; i<n; i++){
        int flag=0;
        for(j=0; j<m; j++){
            if(alloc[i][j]!=0){
                flag=1;
                break;}
        }
        if(flag==0)
            finish[i]=1;
        else
            finish[i]=0;
    }
    int work[m];
    for(i=0; i<m; i++)
        work[i]=avail[i];

    for(k<=0; k<n; k++){
        for(i=0; i<n; i++){
            if(finish[i]==0){
                int canExecute=1;
                for(j=0; j<m; j++){
                    if(request[i][j]>work[j]){
                        canExecute=0;
                        break;}
                }
```

```
if (canExecute) {
    for (j=0; j<m; j++)
        work[j] += alloc[i][j];
    finish[i] = 1; 3 3 3 3
```

```
int deadlock = 0;
printf("\nDeadlocked processes:");

for (i=0; i<n; i++) {
    if (finish[i] == 0) {
        printf("P%d", i);
        deadlock = 1; } }
```

```
if (deadlock == 0)
    printf("None (No Deadlock)\n");
else
    printf("\n System is in
        deadlock\n");
return 0;
}
```

Output

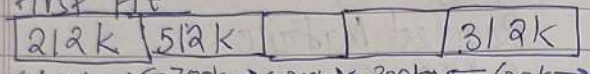
```
Enter no. of processes: 3
Enter no. of resources: 3

Allocation Matrix:
0 1 0
2 0 0
3 0 3
Request Matrix:
0 0 0
2 0 2
0 0 1
Available:
0 0 0

Deadlocked processes: P1 P2
System is in Deadlock
```

5/5/26

Memory Allocation

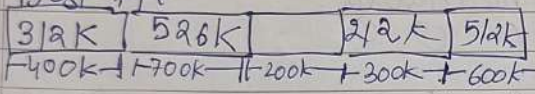
Q1 Given a free memory partition 400K, 700K, 200K, 300K & 600K how would each of first fit, best fit and worst fit algorithms work to place processes of 212K, 512K, 312K, 526K

First Fit



526K cannot be alloted.

Best Fit

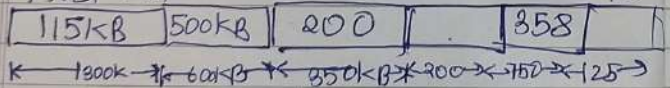


Worst Fit

526K cannot be allocated with memory

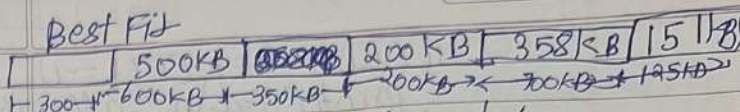
Q2 MP → 300kB, 600kB, 350kB, 200kB, 750kB, 125kB
P → 115kB, 500kB, 358kB, 200kB, ~~375kB~~ 375kB

A-2 First Fit



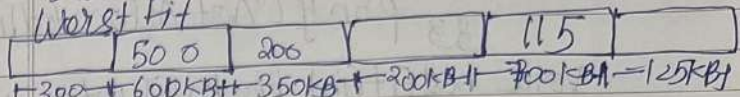
375kB Cannot be alloted

Best Fit



375 cannot be allocated

Worst Fit



Code

```
#include<stdio.h>
#define MAX 100
void firstFit(int blocks[], int m,
int processes[], int n) {
  int allocation[MAX];
  for(int i=0; i<n ; i++)
    allocation[i]= -1;
  for(int i=0; i<n; i++) {
    for(int j=0; j<m; j++) {
      if(blocks[j]>=processes[i]){
        allocation[i]=j;
        blocks[j]=processes[i];
        break;
      }
    }
  }
  printf("\nFirst Fit Allocation:\n");
  printf("Process No.\tProcess Size
  \tBlock No.\n");
  for(int i=0; i<n ;i++) {
    printf("%d\t\t%d\t\t",i+1,
    processes[i]);
```

```
        if (allocation[i] != -1)
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
    }
}
void bestFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++) {
        int bestIdx = -1;
        for(int j=0; j<m; j++) {
            if(blocks[j] >= processes[i]) {
                if(bestIdx == -1 ||
blocks[j] < blocks[bestIdx])
                    bestIdx = j;
            }
        }
        if(bestIdx != -1) {
            allocation[i] = bestIdx;
            blocks[bestIdx] -= processes[i];
        }
    }

    printf("\nBest Fit Allocation:\n");
    printf("Process No.\tProcess Size
\tBlock No.\n");
    for(int i=0; i<n; i++) {
        printf("%d\t\t%d\t\t", i+1,
            processes[i]);
```

```
        if (allocation[i] != -1)
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
    }
}
void worstFit(int blocks[], int m,
int processes[], int n) {
    int allocation[MAX];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++) {
        int worstIdx = -1;
        for(int j=0; j<m; j++) {
            if(blocks[j] >= processes[i]) {
                if(worstIdx == -1 || blocks[j]
>blocks[worstIdx])
                    worstIdx = j;
            }
        }
        if(worstIdx != -1) {
            allocation[i] = worstIdx;
            blocks[worstIdx] -= processes[i];
        }
    }
    printf("\nWorst Fit Allocation:\n");
    printf("Process No.\tProcess Size
\tBlock No.\n");
    for(int i=0; i<n; i++) {
        printf("%d\t\t%d\t\t",i+1,
processes[i]);
```

```
    if(allocation[i] != -1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not Allocated\n");
}
int main() {
    int blocks[MAX], processes[MAX];
    int m, n;
    printf("Enter no. of memory blocks: ");
    scanf("%d", &m);
    printf("Enter size of blocks:\n");
    for(int i=0; i<m; i++)
        scanf("%d", &blocks[i]);
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("Enter size of processes:\n");
    for(int i=0; i<n; i++)
        scanf("%d", &processes[i]);

    int blocks1[MAX], blocks2[MAX],
        blocks3[MAX];
    for(int i=0; i<m; i++) {
        blocks1[i] = blocks[i];
        blocks2[i] = blocks[i];
        blocks3[i] = block[i];
    }
    firstFit(blocks1, m, processes, n);
    bestFit(blocks2, m, processes, n);
    worstFit(blocks3, m, processes, n);
    return 0;
}
```

Output

```
Enter no. of memory blocks: 5
Enter sizes of memory blocks:
400 700 200 300 600
Enter no. of processes: 4
Enter sizes of processes:
212 512 312 526
First Fit Allocation:
1      212      1
2      512      2
3      312      5
4      526      Not Allocated
Best Fit Allocation:
1      212      4
2      512      5
3      312      1
4      526      2
Worst Fit Allocation:
1      212      2
2      512      5
3      312      1
4      526      Not Allocated
```

Q. Simulate Page Replacement Algorithms
① FIFO
② LRU
③ Optimal

A
```
#include<stdio.h>
int findLRU(int time[], int n){
    int i, min=time[0], pos=0;
    for(i=1; i<n; ++i){
        if(time[i]<min){
            min=time[i];
            pos=i;}}
    return pos;
}
int main(){
    int pages[30], frames[10], time[10];
    int n, f, i, j, k, faults, counter,
    pos, found;
    printf("Enter no. of pages: ");
    scanf("%d", &n);
    printf("Enter page reference
      string:");
    for(i=0; i<n; i++)
        scanf("%d", &pages[i]);
    printf("Enter no. of frames:");
    scanf("%d", &f);
```

```
// FIFO
for(i=0; i<f; i++) frames[i]=-1;
faults=0; pos=0;
printf("\nFIFO Page Replacement\n");
for(i=0; i<n; i++){
    found=0;
    for(j=0; j<f; j++)
        if(frames[j]==pages[i])
            found=1;
    if(!found){
        frames[pos]=pages[i];
        pos=(pos+1)%f;
        faults++;}
    printf("Page %d ->", pages[i]);
    for(k=0; k<f; k++)
        (frames[k]!=-1)? printf("%d",
            frames[k]): printf("-");
    printf("\n");}
printf("Total Page Faults=%d\n", faults);

// LRU
for(i=0; i<f; i++) frames[i]=-1;
faults=0; counter=0;
printf("\nLRU Page Replacement:\n");
for(i=0; i<n; i++){
    found=0;
    for(j=0; j<f; j++){
        if(frames[j]==pages[i]
```

```
found = 1;
counter++;
time[j] = counter;
break;
}
if(!found){
  if(i < f)
    frames[i] = pages[i];
  else{
    pos = findLRU(time, f);
    frames[pos] = pages[i];
  }
  counter++;
  time[i%f] = counter;
  faults++;
}
printf("Page %d ->", pages[i]);
for(k=0; k<f; k++)
  (frames[k]==-1) ? printf("%d ", frames[k]) : printf("");
printf("\n");
}

// Optimal
for(i=0; i<f; i++) frames[i] = -1;
faults = 0;
printf("\n Optimal Page Replacement\n");
for(i=0; i<n; i++){
  found = 0;
  for(j=0; j<f; j++)
    if(frames[j]==pages[i]) found = 1;
  if(!found){
    if(i<f)
      frames[i] = pages[i];
    else{
      int future[10], flag[10];
      for(j=0; j<f; j++){
        flag[j] = 0;
        for(k=i+1; k<n; k++){
          if(frames[j]==pages[k]){
            future[j] = k;
            flag[j] = 1;
            break;
          }
        }
        if(!flag[j]) future[j] = n+1;
      }
      int max = future[0];
      pos = 0;
      for(j=1; j<f; j++){
        if(future[j] > max){
          max = future[j];
          pos = j;
        }
      }
      frames[pos] = pages[i];
    }
    faults++;
  }
  printf("Total Page faults = %d", faults);
  return 0;
```

Page No.
Date

output

Enter no. of pages: 20
Enter page reference string
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter no. of frames: 4
FIFO Page Replacement:
Page 7 → 7 - - -
Page 0 → 7 0 - -
Page 1 → 7 0 1 -
Page 2 → 7 0 1 2
Page 0 → 7 0 1 2
Page 3 → 3 0 1 2
Page 0 → 3 0 1 2
Page 4 → 3 4 1 2
Page 2 → 3 4 1 2
Page 3 → 3 4 1 2
Page 0 → 3 4 0 2
Page 3 → 3 4 0 2
Page 2 → 3 4 0 2
Page 1 → 3 4 0 1
Page 2 → 2 4 0 1
Page 0 → 2 4 0 1
Page 1 → 2 4 0 1
Page 7 → 2 7 0 1
Page 0 → 2 7 0 1
Page 1 → 2 7 0 1
Total Page Faults = 10

Page No.
Date

LRU Page Replacement:
Page 7 → 7 - - -
Page 0 → 7 0 - -
Page 1 → 7 0 1 -
Page 2 → 7 0 1 2
Page 0 → 7 0 1 2
Page 3 → 3 0 1 2
Page 0 → 3 0 1 2
Page 4 → 4 0 1 2
Page 2 → 4 0 1 2
Page 3 → 3 0 1 2
Page 0 → 3 0 1 2
Page 3 → 3 0 1 2
Page 2 → 3 0 1 2
Page 1 → 3 0 1 2
Page 2 → 3 0 1 2
Page 0 → 3 0 1 2
Page 7 → 7 0 1 2
Page 0 → 7 0 1 2
Page 1 → 7 0 1 2

Total Page Faults = 8

Page No.
Date

Optimal Page Replacement

Page 7 → 7 - - -
Page 0 → 7 0 - -
Page 1 → 7 0 1 -
Page 2 → 7 0 1 2
Page 0 → 7 0 1 2
Page 3 → 3 0 1 2
Page 0 → 3 0 1 2
Page 4 → 3 0 4 2
Page 2 → 3 0 4 2
Page 3 → 3 0 4 2
Page 0 → 3 0 4 2
Page 3 → 3 0 4 2
Page 2 → 3 0 4 2
Page 1 → 1 0 4 2
Page 2 → 1 0 4 2
Page 0 → 1 0 4 2
Page 1 → 1 0 4 2
Page 7 → 1 0 7 2
Page 0 → 1 0 7 2
Page 1 → 1 0 7 2

Total Page Faults = 8

27/5/25